Computer Architecture Assignment 2
Name: Jai
Roll No: 2301420045

Section A: Core Concepts

CPU Registers: CPU registers are high-speed, small-capacity storage locations directly within the processor. They are critical in design because they provide the fastest possible data access for the ALU, minimizing the need to access slower main memory (RAM) during instruction execution.

Program Counter (PC): The function of the PC is to hold the memory address of the next instruction to be fetched and executed. It automatically increments after each fetch to point to the subsequent instruction in the sequence.

Instruction Format: An instruction format is a bit-level layout that defines how an

instruction is structured, typically consisting of an Opcode (operation to perform) and one or more Operands (data or addresses involved).

Instruction Encoding: Encoding is vital because it determines how efficiently the hardware can decode and execute commands. Proper encoding balances instruction density (to save memory) with decoding simplicity (to increase speed).

Addressing Modes: These are techniques used to specify the location of an operand. Their purpose is to provide flexibility in programming, allowing for features like pointers, arrays, and complex data structures by varying how memory addresses are calculated.

Instruction Set Architecture (ISA): An ISA is the abstract interface between hardware and software. It defines the supported data types, registers, memory

architecture, and the set of instructions that a processor can execute.

Major Instruction Types:

Data Processing: Arithmetic and logical operations.
Data Transfer: Moving data between registers and memory (Load/Store).
Control Flow: Branching, jumping, and procedure calls.
I/O Instructions: Communicating with peripheral devices.

Instruction Decoding: During decoding, the Control Unit interprets the opcode within the Instruction Register (IR) to determine which operation to perform and identifies the required operands, generating the necessary control signals for the hardware.

Basic Instruction Cycle: The cycle consists of three primary stages: Fetch (getting the instruction from memory), Decode

(interpreting the instruction), and Execute (performing the operation).

RISC Characteristics: Reduced Instruction Set Computers feature simple instructions of uniform length, a large number of registers, a load/store architecture, and single-cycle execution for most instructions.

CISC Characteristics: Complex Instruction Set Computers feature a large set of varied instructions, variable-length instruction formats, many addressing modes, and instructions that can perform complex tasks (like memory-to-memory operations) over multiple cycles.

RISC-V vs. x86: RISC-V is an open-standard, modular architecture with a clean, fixed-length instruction design optimized for efficiency. x86 is a proprietary, complex architecture with variable-length instructions and a massive legacy software ecosystem, optimized for

high-performance desktop and server computing.

Section B: Visual Representations

CPU Register Organization
Register
Function
PC (Program Counter)
Holds the address of the next instruction to be fetched.
IR (Instruction Register)
Stores the actual instruction currently being decoded or executed.

MAR (Memory Address Register) - Holds the memory address from which data will be read or to which data will be written.

MBR (Memory Data Register) - Holds the actual data being transferred to or from memory.

ACC (Accumulator) - A general-purpose

register used to store intermediate ALU results.

Instruction Cycle Flow

Fetch: The address in the PC is sent to the MAR. The instruction is retrieved from memory into the MDR and then moved to the IR. The PC is incremented.

Decode: The Control Unit reads the opcode in the IR and activates the necessary internal circuitry (ALU paths, registers).

Execute: The ALU performs the specified operation on operands. Results are stored back in a register or memory.

Instruction Format Structure

Field

Bit Length

Description

Opcode

Bits [15:12]

Defines the operation (e.g., ADD, SUB, LOAD).

Operand 1
Bits [11:6 Specifies the first source register or memory address.
Operand 2
Bits [5:0] – Specifies the second source or destination location.

RISC vs. CISC Comparison
Feature
RISC (e.g., RISC-V)
CISC (e.g., x86)
Instruction Size
Fixed (Uniform)
Variable
Complexity
Simple instructions

Complex, multi-step instructions
Registers
Many general-purpose registers
Fewer, often specialized registers
Execution
Single-cycle (Pipelined)
Multi-cycle
Memory Access
Load/Store only
Memory-to-memory allowed

Section C: Case-Based Analytical Questions

Memory vs. Register Performance: Using memory addresses instead of registers will significantly degrade performance. Memory access is hundreds of times slower than register access, leading to a "memory wall" where the CPU remains idle while waiting for data (increased latency).

Design Choice for Pipelining: The designer should choose RISC. Simple, uniform instruction formats make it much easier to

overlap stages of execution (pipelining) because the hardware can predict the timing and resources needed for every instruction more accurately.

Execution Flow of Two-Operand Instruction:

Fetch: Instruction is moved from memory to IR.
Decode: Hardware identifies the opcode and the two specific registers.
Operand Read: The values from the two registers are sent to the ALU inputs.
Execute: The ALU performs the operation.

Writeback: The final result is saved into the designated destination register.

x86 Longevity: x86 remains dominant primarily due to backward compatibility and a massive software ecosystem. Rebuilding all modern operating systems and applications for a different architecture is

incredibly costly, and modern x86 chips use "micro-ops" to translate complex instructions into RISC-like operations internally to maintain performance.

Section B: One Diagram + One Insight (OBONI)

Task 1: Instruction Cycle Detail (The "Fetch" Stage)

The Fetch stage is the critical starting point. The CPU places the content of the Program Counter (PC) onto the address bus. The memory returns the instruction at that address via the data bus into the Memory Data Register (MDR). Finally, it is moved to the Instruction Register (IR) so the PC can be incremented for the next cycle.

Task 2: RISC-V Instruction Format

A standard RISC-V R-type instruction uses the following fields:

Opcode: Specifies the base operation.
rd: The destination register.
rs1 / rs2: The two source registers.
funct3 / funct7: Additional bits to define specific variations of an operation (e.g., distinguishing between ADD and SUB).

Task 3: Register Architecture and ALU Data Flow

In a standard CPU, the Register File acts as the source for the ALU. Data flows from registers into the ALU via internal buses. Once the ALU performs the calculation, the result is sent back to the Accumulator or a specific destination register in the Register File. The Status Register (Flags) is updated by the ALU to reflect results like "zero" or "overflow."